

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CLASS SCHEDULE

GROUP1: ICE 22-01, 02, 03 MON 11.00 – 12.30 R. No. A202
TUE 09.30 – 11.00 R. No. A202

GROUP2: ICE 22-04, CSE 01,02,03 TUE 11.00 – 12.30 R. A202
THU 11.00 – 12.30 R. A202

GROUP3: CSE 04, 05, 06 WED 09.30 – 11.00 R. No. A202
THU 09.30 – 11.00 R. No. A202

GROUP4: CSE 07, 08, 09 MON 09.30 – 11.00 R. No. A202
WED 11.00 – 12.30 R. No. A202

OFFLINE CLASS RULES

TO BE FOLLOWED VERY STRICTLY

- 1. BE PRESENT IN THE CLASS ATLEAST 5 MINUTES BEFORE THE START OF THE CLASSES**
- 2. STUDENTS LATE BY MORE THAN 5 MINUTES WILL NOT BE ALLOWED TO ENTER THE CLASS**
- 3. USE OF CELLPHONES & LAPTOPS STRICTLY PROHIBITED INSIDE THE CLASS**
- 4. CELL PHONES & LAPTOPS SHOULD NOT BE KEPT ON THE DESK**
- 5. CELLPHONES SHOULD BE KEPT EITHER IN YOUR POCKET OR BACKPACK/BAG YOU CARRY**
- 6. SLEEPING IN CLASS IS TOTALLY PROHIBITED**
- 7. BRING A NOTE BOOK FOR THIS COURSE AND MAKE NOTES IN THIS BOOK DURING THE CLASS**
- 8. NO DRINKING & NO EATING IS ALLOWED IN CLASS (PLEASE NOTE THAT CLASSROOM IS NOT CANTEEN)**

SOC 2040 - SYSTEMS PROGRAMMING

TEAM FORMATION INSTRUCTIONS FOR LAB ASSIGNMENTS

- **ALL LAB ASSIGNMENTS ARE TO BE COMPLETED IN GROUPS OF 4 TO 6 STUDENTS.**
- **YOU ARE REQUIRED TO FORM TEAMS COMPRISING A TEAM LEADER AND MEMBERS FOR CARRYING OUT LAB ASSIGNMENT1.**
- **A TEAM CAN HAVE A MAXIMUM OF 6 MEMBERS.**
- **A TEAM CAN HAVE MEMBERS DRAWN FROM OTHER SECTIONS ALSO.**
- **TEAM LEADER HAS TO PROVIDE THE TEAM DETAILS USING THE TEMPLATE PROVIDED AT THE ECLASS.**
- **DULY-FILLED IN TEAM DETAILS MUST BE SUBMITTED BY EACH TEAM LEADER BY EMAIL TO MY OFFICIAL ACCOUNT n.abdulrahim@inha.uz BY MONDAY, 19TH FEBRUARY 2024.**
- **LATE SUBMISSIONS ARE NOT ENTERTAINED**
- **ADHERE TO THE DEADLINE STRICTLY**

SOC 2040

SYSTEM PROGRAMMING

WEEK 1 LECTURE2

INTRODUCTION TO COMPUTER SYSTEM

Lecture Outline

- ❑ **Computer System -Components**
- ❑ **Programmer's View of a Computer System**
- ❑ **Tour of Computer Systems**
- ❑ **Reading Assignment**



Computer System

- ❑ **Definition - A digital computer**
- ❑ **A contemporary computer is a fast electronic calculating machine that**
 - **accepts digitized input information,**
 - **processes it according to a list of internally stored instructions, and**
 - **produces the resulting output information.**
- **The list of instructions is called a computer program and the internal storage is called computer memory.**

Components of a Computer System

- ★ **Processor**

- Datapath
- Control

- ★ **Memory & Storage**

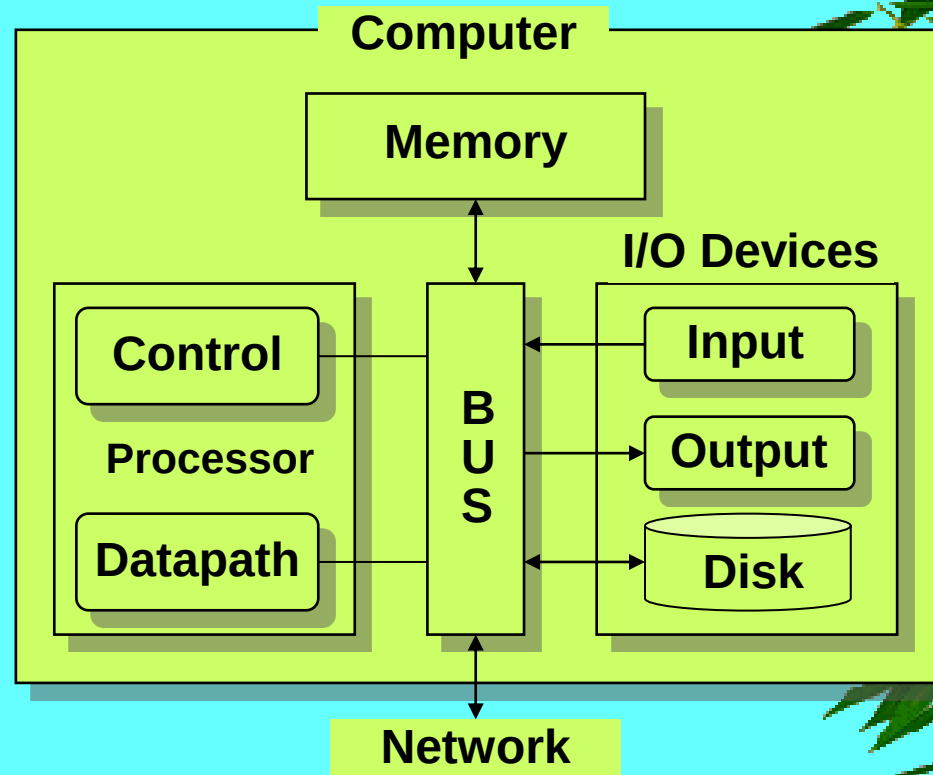
- Main Memory
- Disk Storage

- ★ **Input devices**

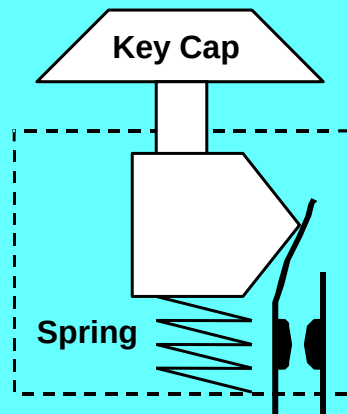
- ★ **Output devices**

- ★ **Bus:** Interconnects processor to memory and I/O

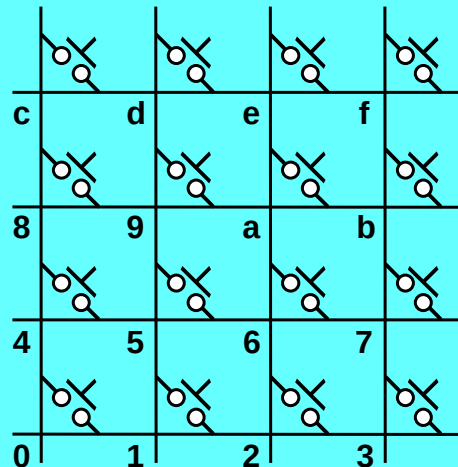
- ★ **Network:** newly added component for communication



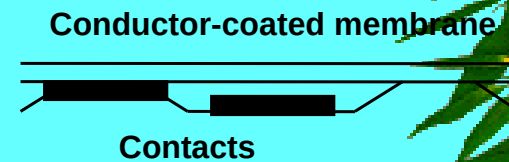
Input Devices



Mechanical switch

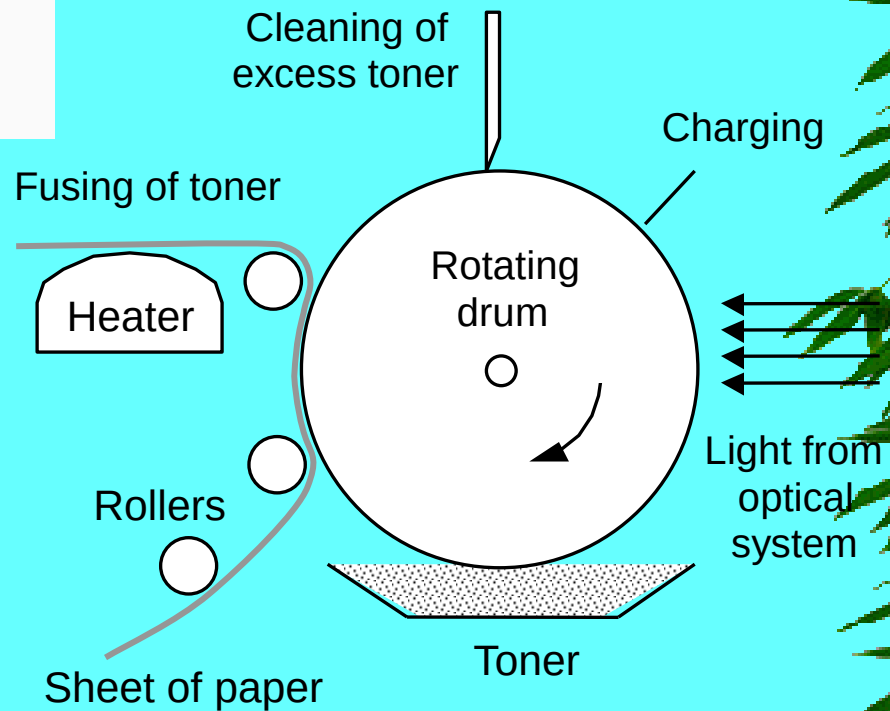
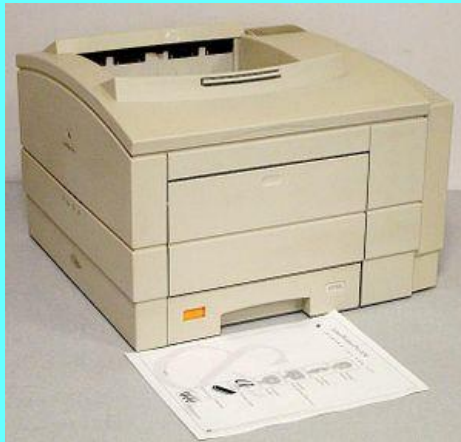


Logical arrangement of keys



Membrane switch

Output Devices

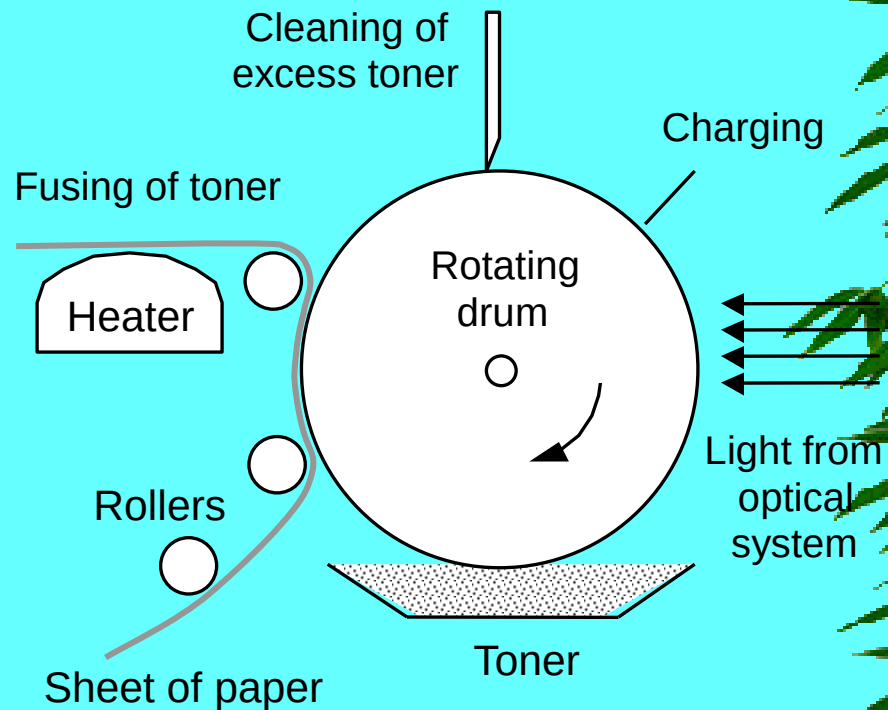


Laser printing

Output Devices

In general the principle of electrostatic laser printing is as follows:

- Charging a photoconductive selenium (or other) coated drum.
- Discharging the drum with the laser steering engine in accordance with the input image rasterized pattern. (the laser is modulated to generate a predefined pixel pattern on the face of the drum - the focal plane).
- The rotating drum attracts toner to the charged pattern (latent image) generated by the laser.
- The toner is transferred from the drum to the moving paper to generate a full image.
- The paper carrying the toner moves through the heater to fuse the toner to a fine non-erasable image.



Memory Devices

★ Volatile Memory Devices

- **RAM** = Random Access Memory
- **DRAM** = Dynamic RAM



- ★ 1-Transistor cell + trench capacitor
- ★ Dense but slow, must be refreshed
- ★ Typical choice for main memory
- **SRAM**: Static RAM
 - ★ 6-Transistor cell, faster but less dense than DRAM
 - ★ Typical choice for cache memory

★ Non-Volatile Memory Devices

- **ROM** = Read Only Memory
- Flash Memory

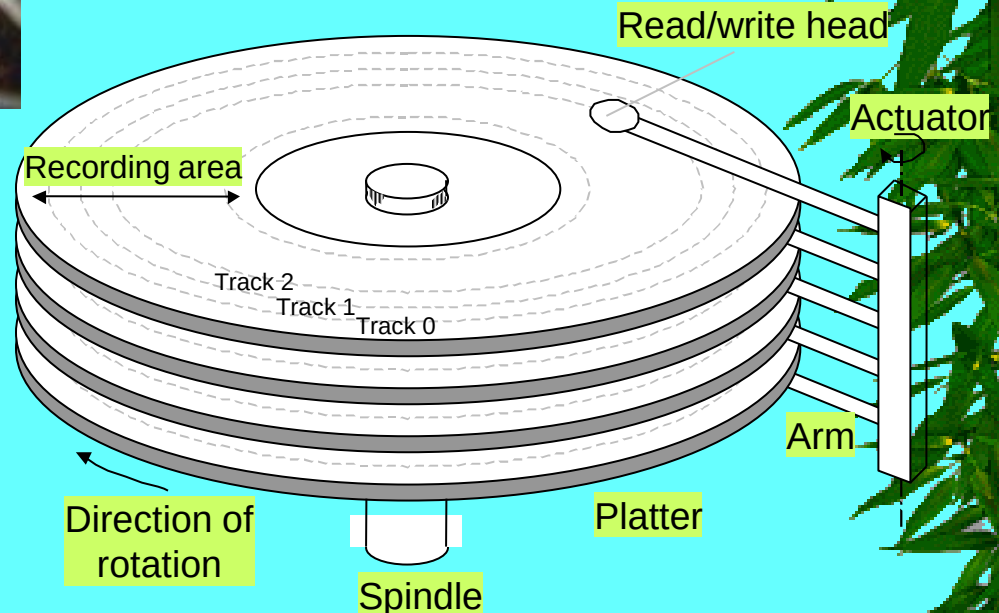


Magnetic Disk Storage



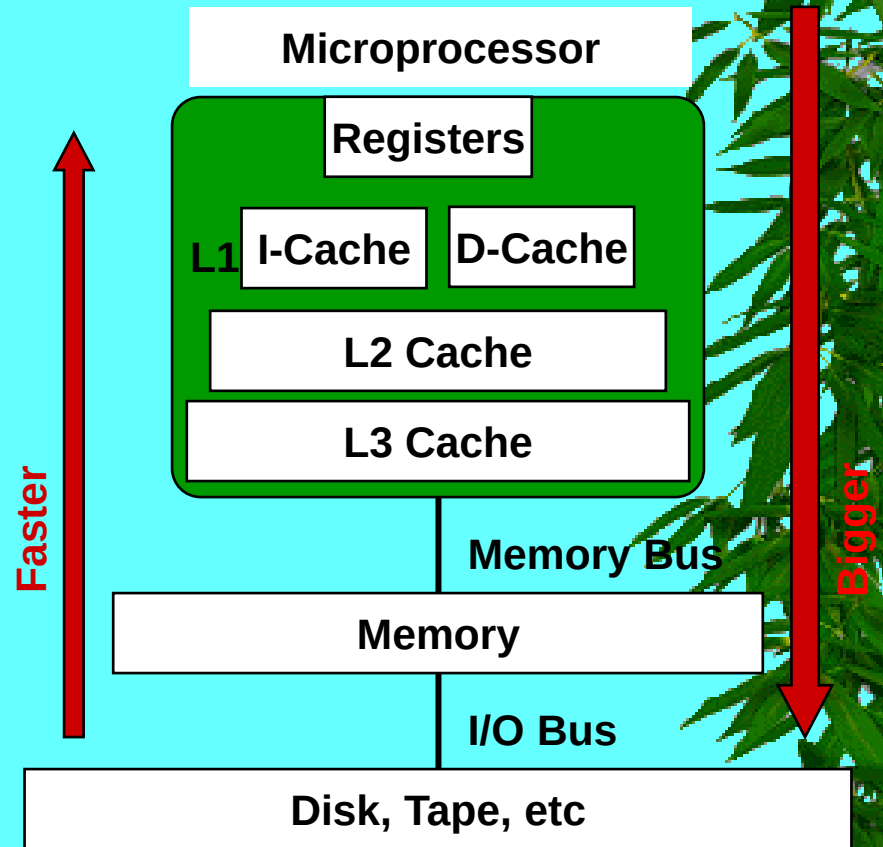
A Magnetic disk consists of a collection of **platters**
Provides a number of **recording surfaces**

Arm provides **read/write heads** for all surfaces
The disk heads are connected together and move in conjunction



Memory Hierarchy

- ★ Registers are at the top of the hierarchy
 - Typically few registers
 - Fastest memory element
- ★ Level 1 Caches (8 – 64 KB)
 - I-Cache stores instructions
 - D-Cache stores data
- ★ L2 Cache (512KB – 4MB)
 - Bigger but slower than L1
- ★ L3 Cache (8MB – 64MB)
 - Bigger but slower than L2
- ★ Main Memory (1 – 8 GB)
 - Access time: 50 – 70 ns
- ★ Disk Storage (> 200 GB)
 - Much slower than main memory, Access time = 5 ms



Address, Data, and Control Bus

★ Address Bus : unidirectional bus

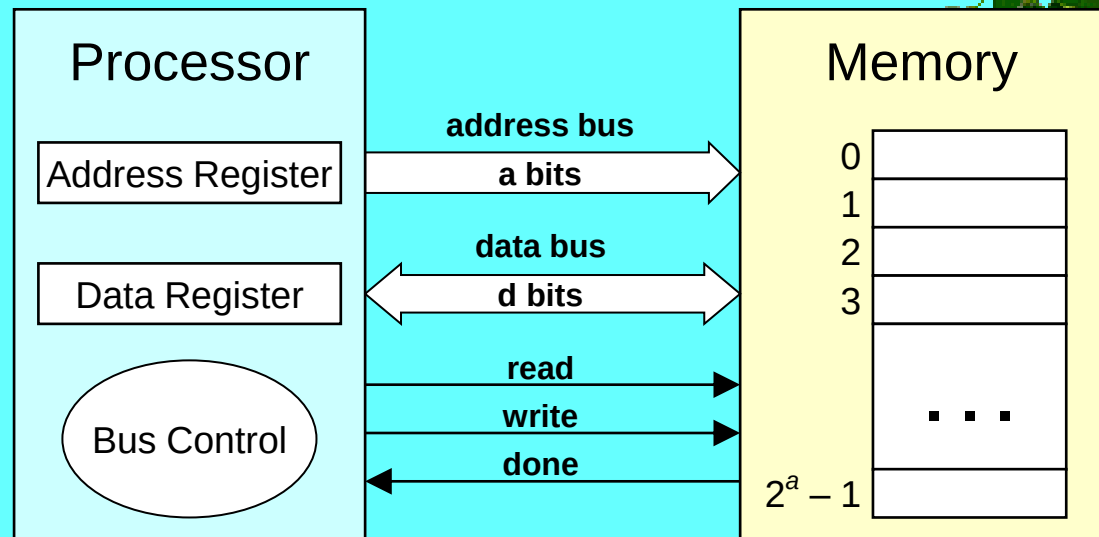
- Memory address is put on address bus
- If memory address = a bits then 2^a locations are addressed

★ Data Bus: bi-directional bus

- Data can be transferred in both directions on the data bus

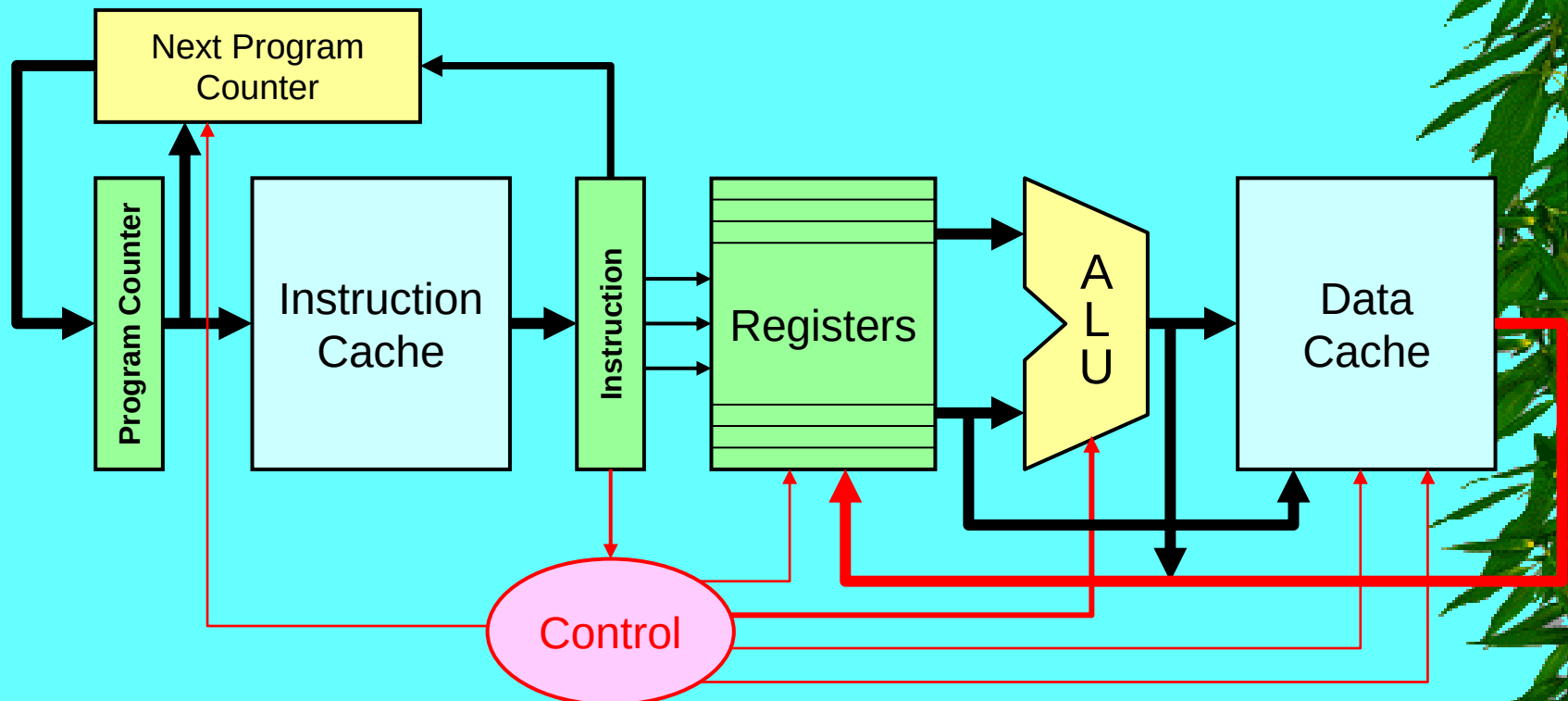
★ Control Bus

- Signals control transfer of data
- Read request
- Write request
- Done transfer



Processor

- ★ **Datapath:** part of a processor that executes instructions
- ★ **Control:** generates control signals for each instruction



Datapath Components

★ Program Counter (PC)

- Contains address of instruction to be fetched
- Next Program Counter: computes address of next instruction

★ Instruction and Data Caches

- Small and fast memory containing most recent instructions/data

★ Register File

- General-purpose registers used for intermediate computations

★ ALU = Arithmetic and Logic Unit

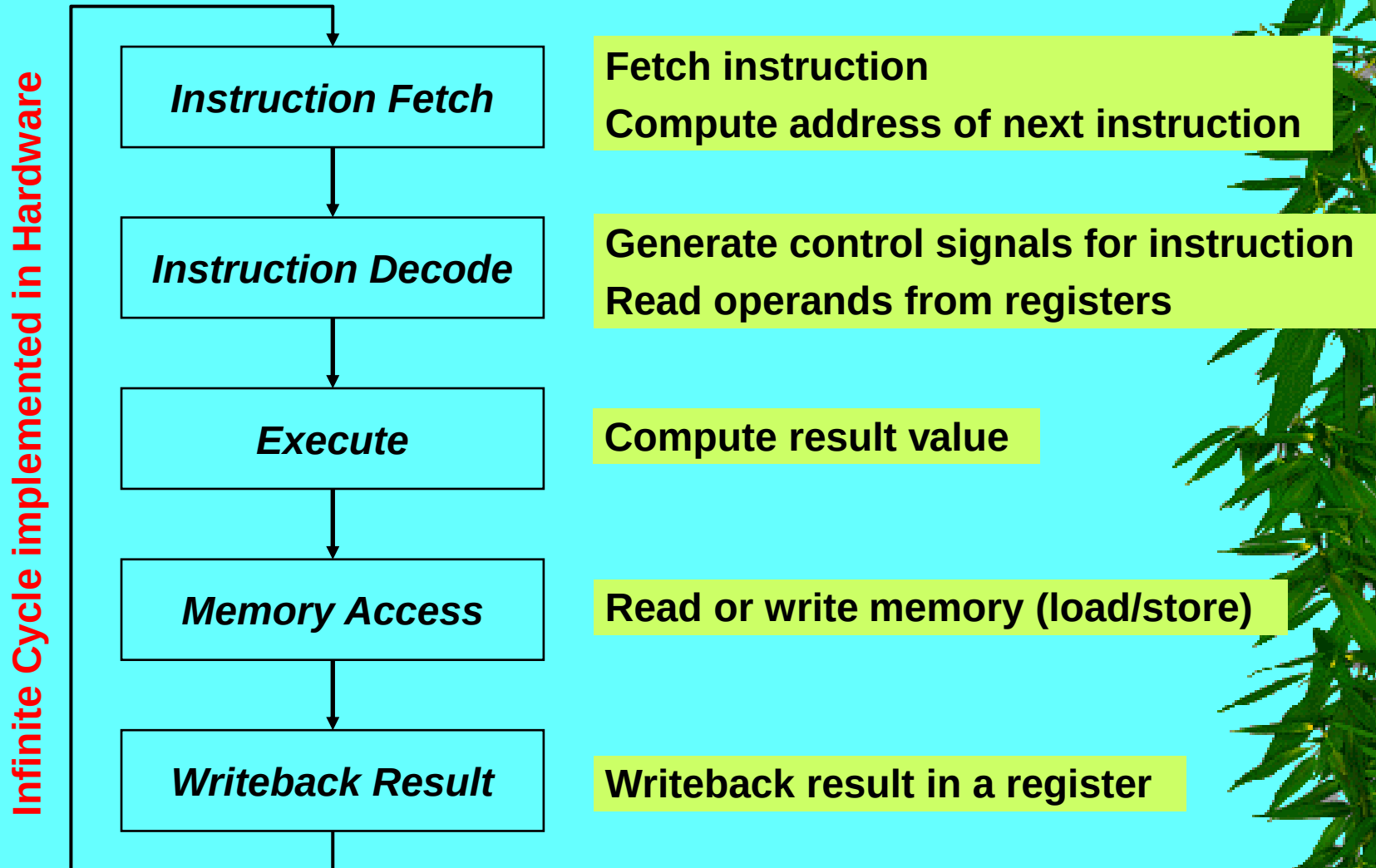
- Executes arithmetic and logic instructions

★ Buses

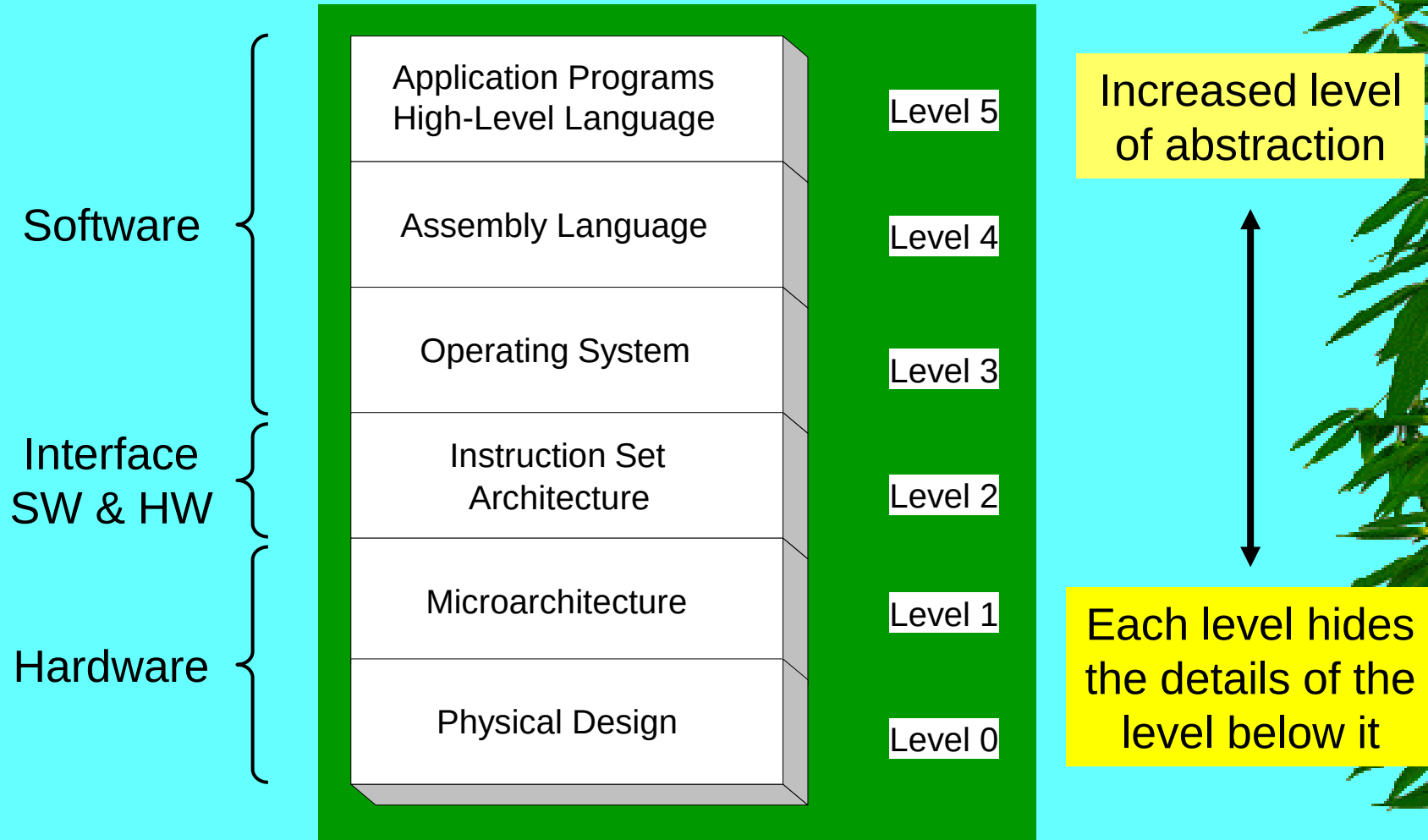
- Used to wire and interconnect the various components



Fetch - Execute Cycle



Programmer's View of a Computer System



Programmer's View of a Computer System

★ Application Programs (Level 5)

- Written in high-level programming languages
- Such as C, Java, C++, Pascal, Visual Basic . . .
- Programs compile into assembly language level (Level 4)

★ Assembly Language (Level 4)

- Instruction mnemonics are used
- Have one-to-one correspondence to machine language
- Calls functions written at the operating system level (Level 3)
- Programs are translated into machine language (Level 2)

★ Operating System (Level 3)

- Provides services to level 4 and 5 programs
- Translated to run at the machine instruction level (Level 2)



Programmer's View of a Computer System

★ Instruction Set Architecture (Level 2)

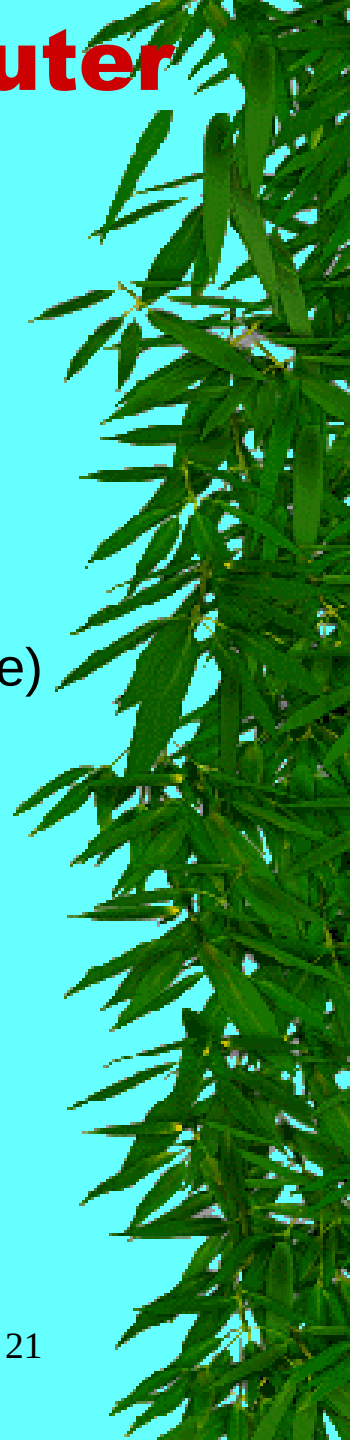
- Interface between software and hardware
- Specifies how a processor functions
- Machine instructions, registers, and memory are exposed
- Machine language is executed by Level 1 (microarchitecture)

★ Microarchitecture (Level 1)

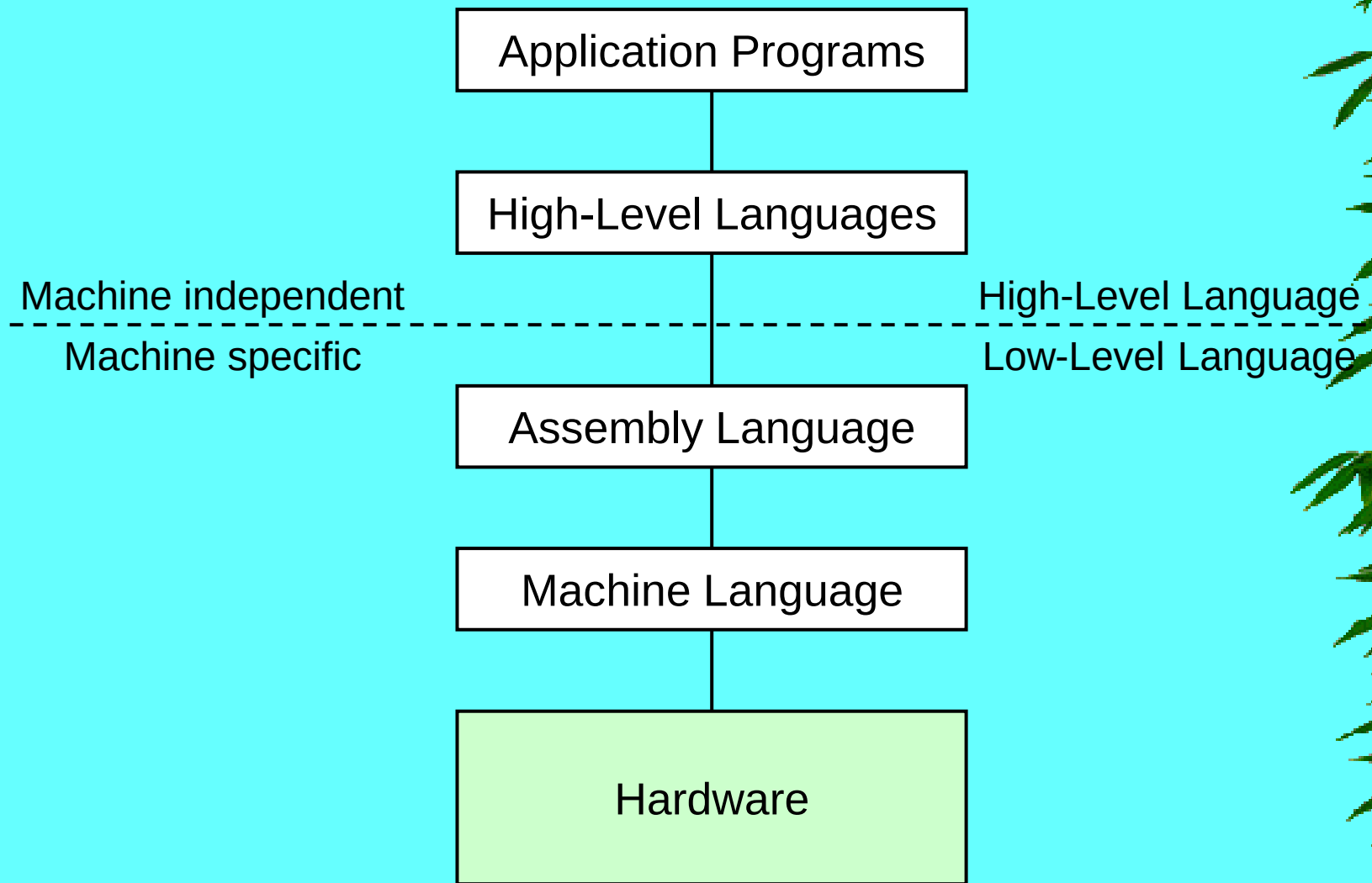
- Controls the execution of machine instructions (Level 2)
- Implemented by digital logic

★ Physical Design (Level 0)

- Implements the microarchitecture
- Physical layout of circuits on a chip



A Hierarchy of Languages



Assembly and Machine Language

★ Machine language

- Native to a processor: executed directly by hardware
- Instructions consist of binary code: 1s and 0s

★ Assembly language

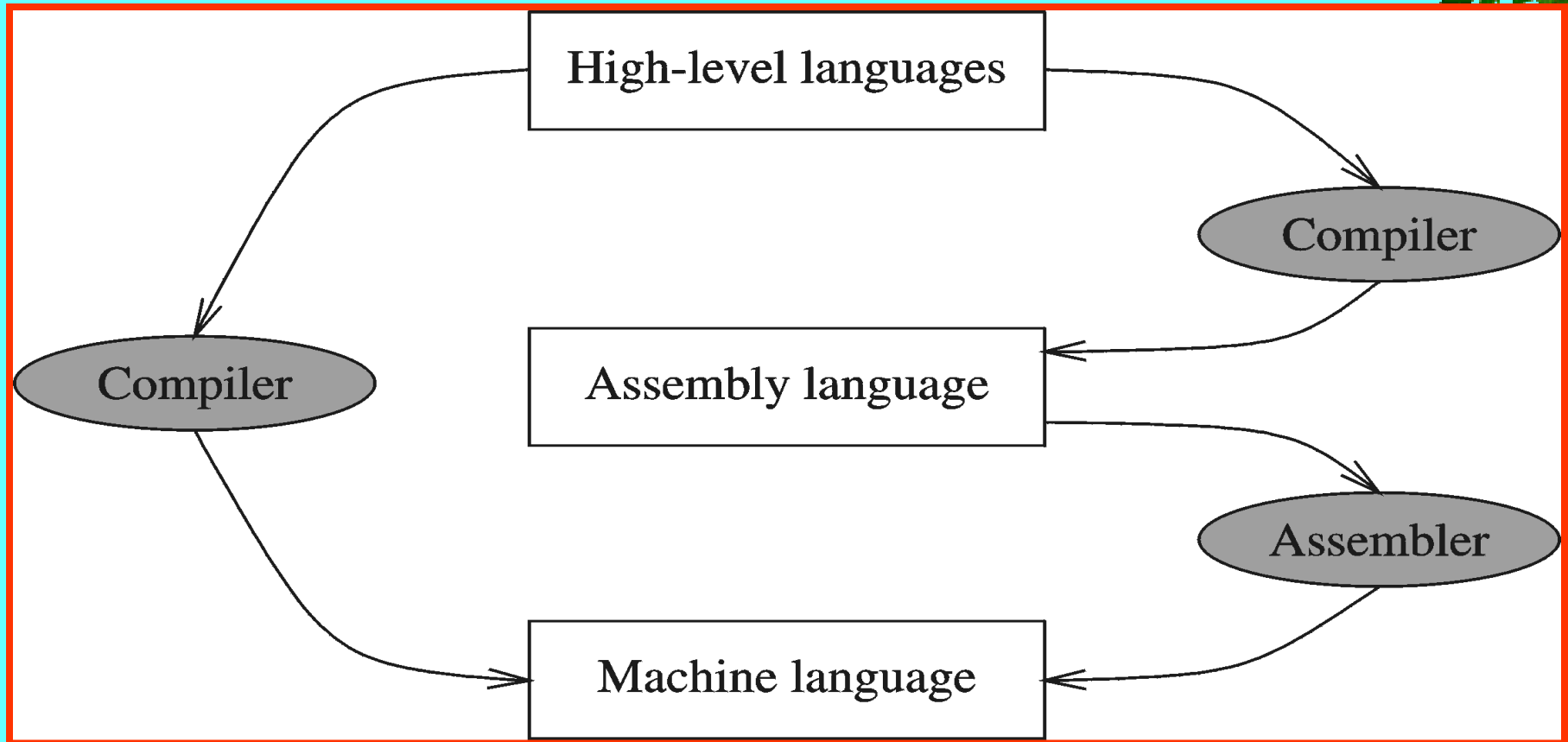
- Slightly higher-level language
- Readability of instructions is better than machine language
- One-to-one correspondence with machine language instructions

★ Assemblers translate assembly language programs to machine code

★ Compilers translate high-level programs to machine code

- Either directly, or
- Indirectly via an assembler

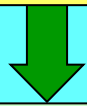
Compiler and Assembler



Translating Languages

Program (C Language):

```
swap(int v[], int k) {  
    int temp;  
    temp = v[k];  
    v[k] = v[k+1];  
    v[k+1] = temp;  
}
```

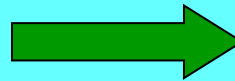


Compiler

MIPS Assembly Language:

```
sll $2,$5, 2  
add $2,$4,$2  
lw  $15,0($2)  
lw  $16,4($2)  
sw  $16,0($2)  
sw  $15,4($2)  
jr  $31
```

Assembler



A statement in a high-level language is translated typically into several machine-level instructions

MIPS Machine Language:

```
00051080  
00821020  
8C620000  
8CF20004  
ACF20000  
AC620004  
03E00008
```

Advantages of High-Level Languages

- ★ **Program development is faster**
 - High-level statements: fewer instructions to code
- ★ **Program maintenance is easier**
 - For the same above reasons
- ★ **Programs are portable**
 - Contain few machine-dependent details
 - ★ Can be used with little or no modifications on different machines
 - Compiler translates to the target machine language
 - However, Assembly language programs are not portable

Why Learn Assembly Language?

- ★ **Many reasons:**

- Accessibility to system hardware
- Space and time efficiency
- Writing a compiler for a high-level language

- ★ **Accessibility to system hardware**

- Assembly Language is useful for implementing system software
- Also useful for small embedded system applications

- ★ **Space and Time efficiency**

- Understanding sources of program inefficiency
- Tuning program performance
- Writing compact code



Assembly Language Programming Tools

★ Editor

- Allows you to create and edit assembly language source files

★ Assembler

- Converts **assembly language** programs into **object files**
- Object files contain the **machine instructions**

★ Linker

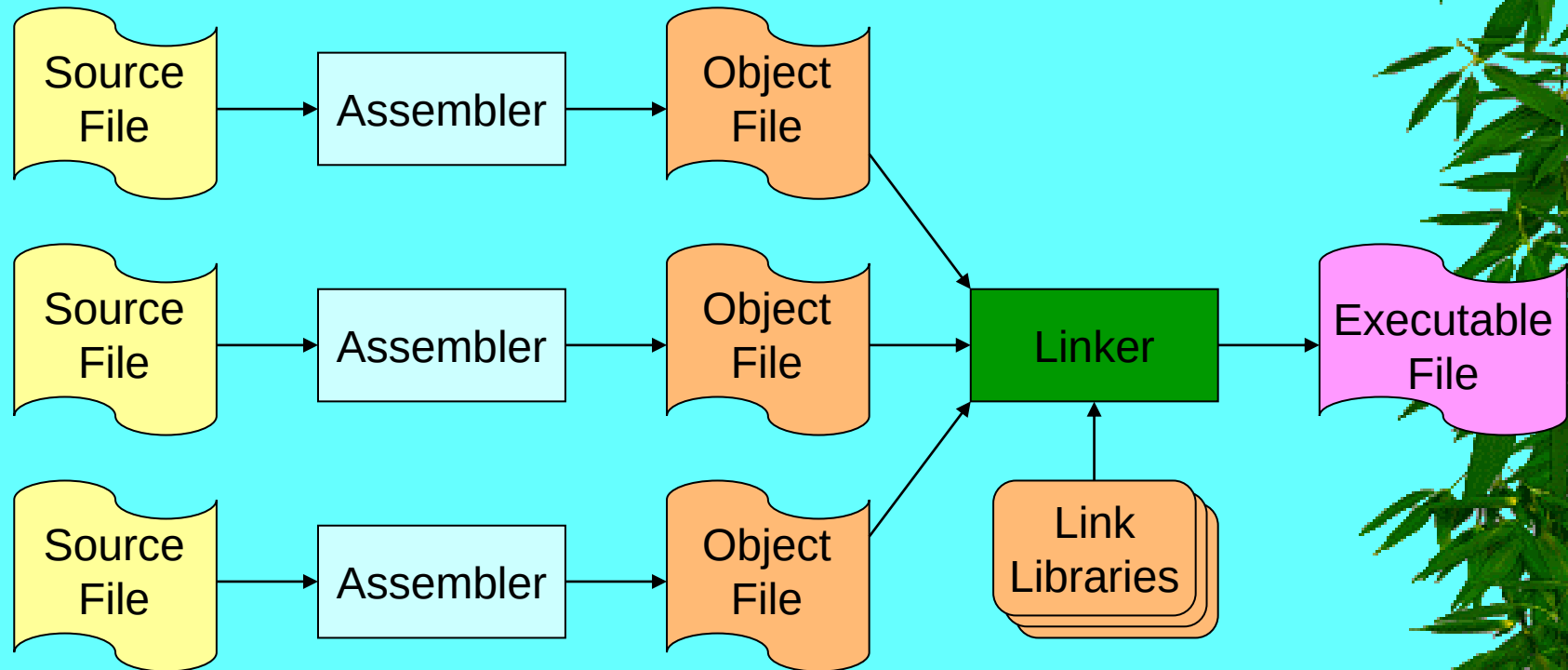
- Combines **object files** created by the assembler with **link libraries**
- Produces a single **executable program**

★ Debugger

- Allows you to trace the execution of a program
- Allows you to view machine instructions, memory, and registers



Assemble and Link Process



A project may consist of multiple source files

Assembler translates each source file separately into an object file

Linker links all object files together with link libraries

SOC 2040

SYSTEM PROGRAMMING

END OF
WEEK 1 LECTURE 2

**INTRODUCTION TO COMPUTER
SYSTEM**

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

